

MAGMAFUNCTIONS MAGMA package documentation

Lucien François 

Version: 03rd of September 2026

Introduction and contents

This document provides documentation for the functions in the package "MAGMAFUNCTIONS.m" that contains functions related to linear algebra, combinatorics and finite fields to be used on MAGMA. This package was build by Lucien FRANÇOIS during its doctoral program in the University College Dublin. The functions have been tested on MAGMA version `Magma V2.28-16`.

1	Help functions	2	4	Linear and multilinear algebra	11
			4.1	Field theory and matrix algebra	11
2	Arithmetics	4	4.2	Tensor algebra	14
3	Combinatorics	5		Summary of functions	15
3.1	Bijections	5			
3.2	Counting in linear algebra	7		Word index	16
3.3	Counting in tensor algebra	8			
3.4	Counting in coding theory	9		Bibliography	17

The references relevant for the mathematical aspects of the functions below are given in the end of the document.



Documentation in progress, sorry for the disturbance.

1 Help functions

<code>MAGMAFUNCTIONS_help()</code>	2	<code>MAGMAFUNCTIONS_help_type(thetype)</code> . .	2
<code>MAGMAFUNCTIONS_info(x)</code>	2	<code>MAGMAFUNCTIONS_info_all()</code>	3

MAGMAFUNCTIONS_help():

Shows the summary of the in-terminal documentation.

```
> MAGMAFUNCTIONS_help();

=====
 1 ARITH      MF_PrimePowersUpTo(m)
 ...

Use MAGMAFUNCTIONS_info(x) for more details on a specific function, where x is
either the function number or the function name. Use
MAGMAFUNCTIONS_help_type(type) for more details on all functions of a specific
type, where type is either ARITH, ...
```

MAGMAFUNCTIONS_info(x):

Displays the documentation for the functions whose numbering or name is x .

```
> MAGMAFUNCTIONS_info(1);

[ARITH] MF_PrimePowersUpTo(m)
      m: positive integer.
Returns the list of prime powers  $p^\eta$  that are less or equal than m.

> MAGMAFUNCTIONS_info("MF_PrimePowersUpTo(m)");

[ARITH] MF_PrimePowersUpTo(m)
      m: positive integer.
Returns the list of prime powers  $p^\eta$  that are less or equal than m.
```

MAGMAFUNCTIONS_help_type(thetype):

Shows the summary of the in-terminal documentation for a specific type `thetype` as displayed in the summary.

```
> MAGMAFUNCTIONS_help_type("BIJCTNS");

=====
31 BIJCTNS    MF_IntToqary(n,q,size)
 ...

Use MAGMAFUNCTIONS_info(x) for more details on a specific function, where x is
either the function number or the function name. Use
MAGMAFUNCTIONS_help_type(type) for more details on all functions of a specific
```

type, where type is either ARITH, CMBTRCS, BIJCTNS, TEST.

MAGMAFUNCTIONS_info_all():

Shows ALL of the in-terminal documentation.

```
> MAGMAFUNCTIONS_all_info();

[ARITH] MF_PrimePowersUpTo(m)
      m: positive integer.
Returns the list of prime powers  $p^{\eta}$  that are less or equal than m.

[LINALG] MF_RandomBasisOfTheField(Fn,F)
      Fn: finite field,
      F: finite field.
Returns a uniformly chosen at random basis of the extension  $F_n$  over  $F$  for  $F_n$  a
finite field extension of  $F$ .
...
...
```

2 Arithmetics

MF_PrimePowersUpTo(m) 4

MF_PrimePowersUpTo(m):

Given an integer m , returns the sequence of prime powers at most m , i.e. returns the sequence of all elements in $\{p^\eta : p \text{ prime}, \eta \in \mathbb{N}, p^\eta \leq m\}$. The asymptotic complexity of the function is the one of the MAGMA function `PrimesUpTo`.

Input	Output
• m : an integer.	A sequence of integers (<code>SeqEnum</code> of <code>RngIntElt</code>).

```
> MF_PrimePowersUpTo(100);
[ 2, 3, 4, 5, 7, 8, 9, 11, 13, 16, 17, 19, 23, 25, 27, 29, 31, 32, 37, 41, 43, 47, 49, 53, 59,
61, 64, 67, 71, 73, 79, 81, 83, 89, 97 ]
```

3 Combinatorics

3.1 Bijections

<code>MF_IntToqary(n, q, size)</code>	5	<code>MF_IntToVSpace(n, q, Vdim)</code>	6
<code>MF_qaryToInt(seq, q)</code>	5	<code>MF_VSpaceToInt(x)</code>	6
<code>MF_IntToFq(n, q)</code>	5	<code>MF_IntToIntTuple(N,i)</code>	6
<code>MF_FqtoInt(x)</code>	6	<code>MF_IntTupleToInt(N,i)</code>	6

`MF_IntToqary(n, q, size):`

Given n a non-negative integer, q an integer with $q \geq 2$, and $size$ a positive integer, returns the image of n through the bijection $\llbracket 0, q^{size} - 1 \rrbracket \xrightarrow{\sim} \llbracket 0, q - 1 \rrbracket^{size}$ that writes the integer n into the basis q . That is the output is $(a_i)_{i \in \llbracket 0, size-1 \rrbracket}$ such that $n = a_0 + a_1q + \dots + a_{size-1}q^{size-1}$.

Input	Output
• n : a non-negative integer, • q : an integer at least 2, • size : a positive integer.	A sequence of integers (<code>SeqEnum</code> of <code>RngIntElt</code>) of size size .

```
> MF_IntToqary(50,2,10);
[ 0, 1, 0, 0, 1, 1, 0, 0, 0, 0 ]
```

`MF_qaryToInt(seq, q):`

Given seq a sequence of non-negative integers and q an integer with $q \geq 2$ and strictly larger than every entry of seq , returns the image of seq through the bijection $\llbracket 0, q - 1 \rrbracket^{size} \xrightarrow{\sim} \llbracket 0, q^{size} - 1 \rrbracket$ such that $(a_i)_{i \in \llbracket 0, size \rrbracket} \mapsto a_0 + a_1q + \dots + a_{size-1}q^{size-1}$, where $size$ is the length of the sequence seq .

Input	Output
• seq : A sequence of integers (<code>SeqEnum</code>), • q : an integer at least $\max(2, \max(seq))$.	An integer (<code>RngIntElt</code>).

```
> MF_qarytoInt([ 0, 1, 0, 0, 1, 1, 0, 0, 0, 0 ],2);
50
```

`MF_IntToFq(n, q):`

....

Input	Output
• ...	An integer (<code>RngIntElt</code>).

MF_FqtoInt(x):

....

Input	Output
• ...	An integer (RngIntElt).

--

MF_IntToVSpace(n, q, Vdim):

....

Input	Output
• ...	An integer (RngIntElt).

--

MF_VSpaceToInt(x):

....

Input	Output
• ...	An integer (RngIntElt).

--

MF_IntToIntTuple(N,i):

MF_IntTupleToInt(N,i):

3.2 Counting in linear algebra

`MF_GBC(q, n, r)` 7 `MF_NbMatrixRank(n, m, q, r)` 7

MF_GBC(q, n, r):

Given a prime power q and two non-negative integers n and r , returns the Gaußian binomial coefficient-Gaußian binomial coefficient

$$\begin{bmatrix} n \\ r \end{bmatrix}_q := \begin{cases} \prod_{t=1}^r \frac{q^{n-t}-1}{q^t-1} & \text{if } r \leq n \\ 0 & \text{else.} \end{cases}$$

That is returns the number of r -dimensional subspaces of the vector-space $\mathbb{F}_q^n$.

Input	Output
• q : a prime power, • n : a non-negative integer, • r : a non-negative integer.	An integer (RngIntElt).

```
> MF_GBC(8,5,3);
304265
```

MF_NbMatrixRank(n, m, q, r):

Given non-negative integers n , m , and r with a prime power q the number of matrices in $\mathbb{F}_q^{n \times m}$ of rank exactly r , that is to say the integer

$$\begin{bmatrix} m \\ r \end{bmatrix}_q \sum_{k=0}^r (-1)^{r-k} \begin{bmatrix} r \\ k \end{bmatrix}_q q^{nk + \binom{r-k}{2}} = \begin{bmatrix} m \\ r \end{bmatrix}_q \prod_{i=0}^{r-1} (q^n - q^i).$$

This functions needs the function `MF_GBC`. See [VLW92, Theorem 25.2].

Input	Output
• n : a non-negative integer, • m : a non-negative integer, • q : a prime power, • r : a non-negative integer.	An integer (RngIntElt).

```
> MF_NbMatrixRank(6,5,3,2);
639518880
```

3.3 Counting in tensor algebra

MF_NbTrankOne(q,N) 8 **MF_NbTrankTwo(q,N)** 8

MF_NbTrankOne(q,N):

Given a prime power q , given $N = [n_1, \dots, n_m]$ a tuple of non-negative integers, returns the number of tensors of size $n_1 \times \dots \times n_m$ over $\mathbb{F}_q$ with rank one, i.e. returns

$$\frac{\prod_{j=1}^m (q^{n_j} - 1)}{(q - 1)^{m-1}}.$$

See [VLW92, Theorem 25.2].

Input	Output
• q : a prime power, • N : a tuple of non-negative integers.	An integer.

```
> MF_NbTrankOne(4, [5, 4, 3, 2]);
3043425
```

MF_NbTrankTwo(q,N):

Given a prime power q , given $N = [n_1, \dots, n_m]$ a tuple of non-negative integers **of length at most 3**, returns the number of tensors of size $n_1 \times \dots \times n_m$ over $\mathbb{F}_q$ with rank two. If $m = 1$, it returns 0. If $m = 2$, it returns $\text{MF_NbMatrixRank}(n_1, n_2, q, 2)$. See [BCF26, Lemma 5.1] for the value for 3-tensors.

Input	Output
• q : a prime power, • N : a (≤ 3)-tuple of non-negative integers.	An integer.

```
> MF_NbTrankOne(4, [5, 4, 3]);
608685
> MF_NbTrankTwo(4, [5, 4, 3]);
1574709485580
```

3.4 Counting in coding theory

MF_MRDrankdistribution(q,m,n,d,r) . . . 9 MF_MRDeigenvaluetrilinearforms(...) 10
MF_MRDrankenumerator(q,m,n,d,x,y) . . . 9

MF_MRDrankdistribution(q,m,n,d,r) :

Given a prime power q , given m, n, r non-negative integers and d a positive integer, returns the number of matrices of rank exactly r in an $[m \times n, \max(m, n)(\min(m, n) - d + 1), d]_q$ maximum rank-distance rank-metric code, that is to say a subspace $\mathcal{C} \leq \mathbb{F}_q^{m \times n}$ of minimum rank d and dimension $\max(m, n)(\min(m, n) - d + 1)$. This functions needs the function MF_GBC. See [Gor21, Theorem 11.4.15].

Input	Output
• q : a prime power, • m : a non-negative integer, • n : a non-negative integer, • d : a postive integer, • r : a non-negative integer.	An integer (RngIntElt).

```
> for r in [0..5] do printf "%o, ", MF_MRDrankdistribution(2,6,5,3,r); end for;
1, 0, 0, 9765, 97650, 154728,
```

MF_MRDrankenumerator(q,m,n,d,x,y):

Given a prime power q , given m, n, r non-negative integers and d a positive integer, returns the value of the rank-weight enumerator of an $[m \times n, \max(m, n)(\min(m, n) - d + 1), d]_q$ maximum rank-distance rank-metric code. In other words, if a_r is the number of rank- r matrices in an $[m \times n, \max(m, n)(\min(m, n) - d + 1), d]_q$ MRD rank-metric code for all $r \in \llbracket 0, \min(m, n) \rrbracket$, the function returns

$$\sum_{r=0}^{\min(m,n)} a_r x^{\min(m,n)-r} y^r.$$

Note that the functions works too if x and y are variables of a polynomial ring. In this case, the output is an homogeneous polynomial of the same polynomial ring. This functions needs the functions MF_GBC and MF_MRDrankdistribution(q, m, n, d, r). See [Gor21, Definition 11.6.1, Theorem 11.4.15].

Input	Output
• q : a prime power, • m : a non-negative integer, • n : a non-negative integer, • d : a postive integer, • x, y : elements of the same ring.	An element of the ring of x and y .

```
> MF_MRDrankenumerator(2,10,8,6,1,1/2);
5238784
> Q<X,Y> := PolynomialRing(RationalField(),2);
> MF_MRDrankenumerator(2,10,8,6,X,Y);
X^8 + 11043285*X^2*Y^6 + 234256770*X*Y^7 + 828441768*Y^8
```

MF_MRDeigenvaluetrilinearforms(q,n1,n2,n3,d,mode):

Given a prime power q , given $n1, n2, n3$ non-negative integers and d a positive integer, returns the eigenvalue the Cayley graph of $\mathbb{F}_q^{n1 \times n2 \times n3}$ generated by its rank-one elements which is associated to a tensor $x \in \mathbb{F}_q^{n1 \times n2 \times n3}$ whose $mode$ -th slice-space is an MRD rank-metric code. In other words, if $\{\{m, n\}\} = \{\{n_j : j \neq mode\}\}$, if a_r is the number of rank- r matrices in an $[m \times n, k, d]_q$ MRD rank-metric code ($k = \max(m, n)(\min(m, n) - d + 1)$) for all $r \in \llbracket 0, \min(m, n) \rrbracket$, the function returns

$$\frac{1}{(q-1)^2} \left((q^{n1} - 1)(q^{n2} - 1)(q^{n3} - 1) - q^{n1+n2+n3} + q^{n1+n2+n3-k} \left(\sum_{r=0}^{\min(m,n)} a_r q^{-r} \right) \right).$$

This functions needs the functions `MF_GBC`, `MF_MRDrankdistribution` and `MF_MRDrankenumerator`. See [\[BF26\]](#).

Input	Output
• q : a prime power, • n1,n2,n3 : non-negative integers, • d : a positive integer, • moe : an integer in $\{1, 2, 3\}$.	An integer.

```
> MF_MRDeigenvaluetrilinearforms(2,10,9,8,4,3);
-522753
```

4 Linear and multilinear algebra

4.1 Field theory and matrix algebra

<code>MF_RandomBasisOfTheField(Fn, F)</code> . . .	11	<code>MF_DualBasis(alpha)</code>	12
<code>MF_Moore(alpha,q : nbcot)</code>	11	<code>MF_Dickson(alpha,q)</code>	12

`MF_RandomBasisOfTheField(Fn, F):`

Given F a finite field and Fn a finite field extension of F , returns a basis of Fn over F chosen uniformly at random.

Input	Output
• Fn : a finite field, • F : a finite field with size dividing the size of Fn.	A sequence of elements in Fn .

```
> F := GF(2);
> Fn := ext<F|4>;
> AssignNames(~Fn,["g"]);
> MF_RandomBasisOfTheField(Fn,F);
[ g^9, g^14, 1, g^8 ]
> MF_RandomBasisOfTheField(Fn,F);
[ g^8, g^9, g^2, g^6 ]
> MF_RandomBasisOfTheField(Fn,F);
[ g^2, 1, g^7, g^13 ]
```

`MF_Moore(alpha,q : nbcot):`

Given $\alpha = [\alpha_1, \dots, \alpha_n]$ a non-empty tuple of elements over a finite field extension of $\mathbb{F}_q$, returns the Moore matrix of the sequence, that is the matrix

$$\begin{bmatrix} \alpha_1 & \alpha_1^q & \cdots & \alpha_1^{q^{k-1}} \\ \alpha_2 & \alpha_2^q & \cdots & \alpha_2^{q^{k-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_n & \alpha_n^q & \cdots & \alpha_n^{q^{k-1}} \end{bmatrix}$$

where k is the degree of $\text{Parent}(\alpha_1)$ over $\mathbb{F}_q$. If $nbcot$ is a specified integer, return instead the Matrix made of the first $nbcot$ columns of the matrix above.

Input	Output
• α : a sequence of elements in a finite field, • q : a prime power dividing the field size of elements in α,	A $\#\alpha \times k$ matrix over the same field as α .

```
> F:= GF(2);
> Fn := ext<F|4>; AssignNames(~Fn,["g"]);
> Ba := MF_RandomBasisOfTheField(Fn,F); Ba;
[ g^11, g^8, g^5, g^6 ]
> MF_Moore(Ba,2);
[g^11 g^7 g^14 g^13]
```

```

[ g^8    g   g^2  g^4]
[ g^5 g^10 g^5 g^10]
[ g^6 g^12 g^9  g^3]
> MF_Moore(Ba,2 : nbc0l := 3);
[g^11 g^7 g^14]
[ g^8    g   g^2]
[ g^5 g^10 g^5]
[ g^6 g^12 g^9]

```

MF_DualBasis(alpha):

Given $alpha = \alpha = [\alpha_1, \dots, \alpha_n]$ a basis of a certain finite field $\mathbb{F}_{q^n}$ over $\mathbb{F}_q$, returns the dual basis $\alpha^\perp = [\alpha_1^\perp, \dots, \alpha_n^\perp]$ of α , that is to say the unique sequence such that we have

$$\forall i, j \in \llbracket 1, n \rrbracket : \text{Tr}_{\mathbb{F}_{q^n}/\mathbb{F}_q}(\alpha_i \alpha_j^\perp) = \sum_{\ell=0}^{n-1} (\alpha_i \alpha_j^\perp)^{q^\ell} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{else.} \end{cases}$$

Input	Output
alpha : a sequence of elements in a finite field, which must be one of its basis.	A sequence with same field and size as alpha .

```

> F := GF(4);
> Fn := GF(4^3); AssignNames(~Fn, ["g"]);
> Ba := MF_RandomBasisOfTheField(Fn, F); Ba;
[ g^10, g^50, g^30 ]
> DuBa := MF_DualBasis(Ba); DuBa;
[ g^6, g^47, g^41 ]
> for i in [1,2,3], j in [1,2,3] do
for> printf "Tr(a_%ob_%o) = %o\n", i, j, Trace(Ba[i]*DuBa[j], F);
for> end for;
Tr(a_1b_1) = 1
Tr(a_1b_2) = 0
Tr(a_1b_3) = 0
Tr(a_2b_1) = 0
Tr(a_2b_2) = 1
Tr(a_2b_3) = 0
Tr(a_3b_1) = 0
Tr(a_3b_2) = 0
Tr(a_3b_3) = 1

```

MF_Dickson(alpha, q):

Given $alpha = [\alpha_1, \dots, \alpha_n]$ a non-empty tuple of elements over a finite field extension of $\mathbb{F}_q$, returns the Dickson matrix of the sequence, that is the matrix

$$\begin{bmatrix} \alpha_1 & \alpha_n^q & \alpha_{n-1}^{q^2} & \cdots & \alpha_2^{q^{n-1}} \\ \alpha_2 & \alpha_1^q & \alpha_n^{q^2} & \cdots & \alpha_3^{q^{n-1}} \\ \alpha_3 & \alpha_2^q & \alpha_1^{q^2} & \cdots & \alpha_4^{q^{n-1}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_n & \alpha_{n-1}^q & \alpha_{n-2}^{q^2} & \cdots & \alpha_1^{q^{n-1}} \end{bmatrix}.$$

Input	Output
• alpha : a sequence of elements in a finite field, • q : a prime power dividing the field size of elements in alpha,	A square matrix of size #alpha over the same field as alpha .

```

> F:= GF(2);
> Fn := ext<F|4>; g := Fn.1; AssignNames(~Fn,["g"]);
> Ba := [g^i : i in [0..3]];Ba;
[ 1, g, g^2, g^3 ]
> MF_Moore(Ba,2);
[ 1 1 1 1]
[ g g^2 g^4 g^8]
[ g^2 g^4 g^8 g]
[ g^3 g^6 g^12 g^9]
> MF_Dickson(Ba,2);
[ 1 g^6 g^8 g^8]
[ g 1 g^12 g]
[ g^2 g^2 1 g^9]
[ g^3 g^4 g^4 1]

```

4.2 Tensor algebra

Summary of Functions

MF_Dickson(alpha,q), 12	MF_Moore(alpha,q : nbc), 11
MF_DualBasis(alpha), 12	MF_MRDeigenvaluetriplelinearforms(...), 10
	MF_MRDrankdistribution(q,m,n,d,r) , 9
MF_FqtoInt(x), 6	MF_MRDrankenumerator(q,m,n,d,x,y), 9
MF_GBC(q, n, r), 7	MF_NbMatrixRank(n, m, q, r), 7
	MF_NbTrankOne(q,N), 8
MF_IntToFq(n, q), 5	MF_NbTrankTwo(q,N), 8
MF_IntToIntTuple(N,i), 6	
MF_IntToqary(n, q, size), 5	
MF_IntToVSpace(n, q, Vdim), 6	MF_PrimePowersUpTo(m), 4
MF_IntTupleToInt(N,i), 6	
	MF_qaryToInt(seq, q), 5
MAGMAFUNCTIONS_help(), 2	
MAGMAFUNCTIONS_help_type(thetype), 2	MF_RandomBasisOfTheField(Fn, F), 11
MAGMAFUNCTIONS_info(x), 2	
MAGMAFUNCTIONS_info_all(), 3	MF_VSpaceToInt(x), 6

Word Index

basis, 11

Cayley graph, 10

Dickson matrix, 12

dual basis, 12

eigenvalue, 10

Gaussian binomial coefficient, 7

maximum rank-distance, 9, 10

Moore matrix, 11

rank-weight enumerator, 9

References

- [BCF26] Eimear Byrne, Alain Couvreur, and Lucien François. Decoding algorithms for tensor codes. *arXiv:2604.16105*, 2026.
- [BF26] Eimear Byrne and Lucien François. The multilinear forms Cayley graph and the eigenvalue method for tensor codes. *arXiv preprint*, 2026.
- [Gor21] Elisa Gorla. Rank-metric codes. In *Concise Encyclopedia of Coding Theory*. Chapman and Hall/CRC Press, 2021.
- [VLW92] J. H. Van Lint and R. M. Wilson. *A Course in Combinatorics*. Cambridge University Press, Cambridge, 1992.